

Syntactic Grammar Design

Expression → Literal | Binary | Unary | Grouping

Literal → NUMBER | STRING | “true” | “false” | “null”

Binary → expression operator expression

Unary → (“-“ | “!”) expression

Grouping → (“(expression)”)

Operator → “==” | “!=” | “<” | “<=” | “>” | “>=” | “+” | “-” | “*” | “/” | “%”

I have decided to maintain Lox’s expression grammar, with the addition of the “%” modulo operator that I included.

Extra Credit Opportunities

No extra credit opportunities were taken in this lab.

Testing

*Note – All test cases have the following code at the beginning:

```
import sys

from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[2] / "src"))
```

This code was generated by AI and each case contains a comment denoting this AI code.

This code creates a Path object of my src folder and adds it to Python’s search path to import the classes I need to run the AST printer. With this code, I now do not need to keep my test files in the src folder.

TC1 – Example Test

Purpose

- This test is to include the example from the lab instructions.

Expected Output: (* (- 123) (group 45.67))

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343\test\Lab2> python TC1-Example.py
(* (- 123) (group 45.67))
```

TC2- Arithmetic

Purpose

- Test all expression types – Literal, Grouping, Binary, Unary
- Test all of the arithmetic operators together
- Test increased nesting

Expected Output: (+ (/ (group (+ (- 10 5) (% (* 7 2) 2))) 1) (- 25))

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343\test\Lab2> python TC2-Arithmetic.py
(+ (/ (group (+ (- 10 5) (% (* 7 2) 2))) 1) (- 25))
```

TC3 -Comparison1

Purpose

- Test the comparison operators

Expected Output: (== (group (> 15 9)) (group (>= 2 7)))

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343\test\Lab2> python TC3-Comparison1.py
(== (group (> 15 9)) (group (>= 2 7)))
```

TC4 – Comparison2

Purpose

- Test the rest of the comparison operators

Expected Output: `(!= (group (<= 4 10)) (group (< 6 8)))`

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343\test\Lab2> python TC4-Comparison2.py  
(!= (group (<= 4 10)) (group (< 6 8)))
```

TC5 – Boolean1

Purpose

- Test the true, false, and null literal

Expected Output: `(or True (group (and (<= hello world) (!= word Null))))`

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343\test\Lab2> python TC5-Boolean1.py  
(or True (group (and (<= hello world) (!= word Null))))
```

TC6- Boolean2

Purpose

- Test the true, false, null literal

Expected Output: `(== (group (! True)) (group (== False Null)))`

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343\test\Lab2> python TC6-Boolean2.py  
(== (group (! True)) (group (== False Null)))
```

TC7 – Decimal Numbers

Purpose

- Test Decimal Literals
- Test increased nesting

Expected Output: `(+ (+ (group (/ (group (* 0.9 2.1)) 1.1)) (- 5.0)) 0.2)`

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343\test\Lab2> python TC7-Decimal_Numbers.py
(+ (+ (group (/ (group (* 0.9 2.1)) 1.1)) (- 5.0)) 0.2)
```

TC8 – Simple

Purpose

- Test each expression group on its own
- Test cases with no nesting

Expected Output:

`(+ 10 5)`

`(== True False)`

`(- 5)`

`950`

`hello`

`(group 5)`

`(group (* 4 2))`

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\desktop\CNeumann_CIS343\CNeumann_CIS343\test\Lab2> python TC8-Simple.py
(+ 10 5)
(== True False)
(- 5)
950
hello
(group 5)
(group (* 4 2))
```